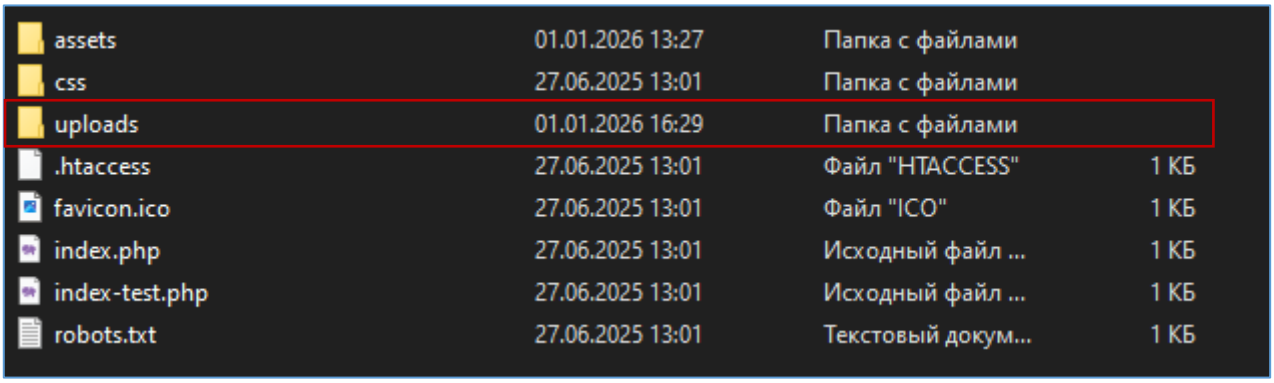
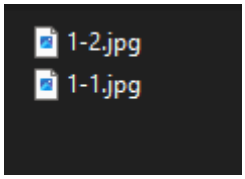
Реализация созданий заявки с проблемой в личном кабинете.

Создадим две категории через панель администратора. Категории: Дороги и ЖКХ.

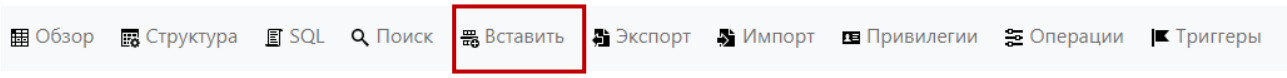
В каталоге web создадим подкаталог uploads для хранения медиафайлов проекта.



Добавим два любых изображения для создания первой заявки. 1-1.jpg – это изображение photoBefore, 1-2.jpg – это изображение photoAfter.

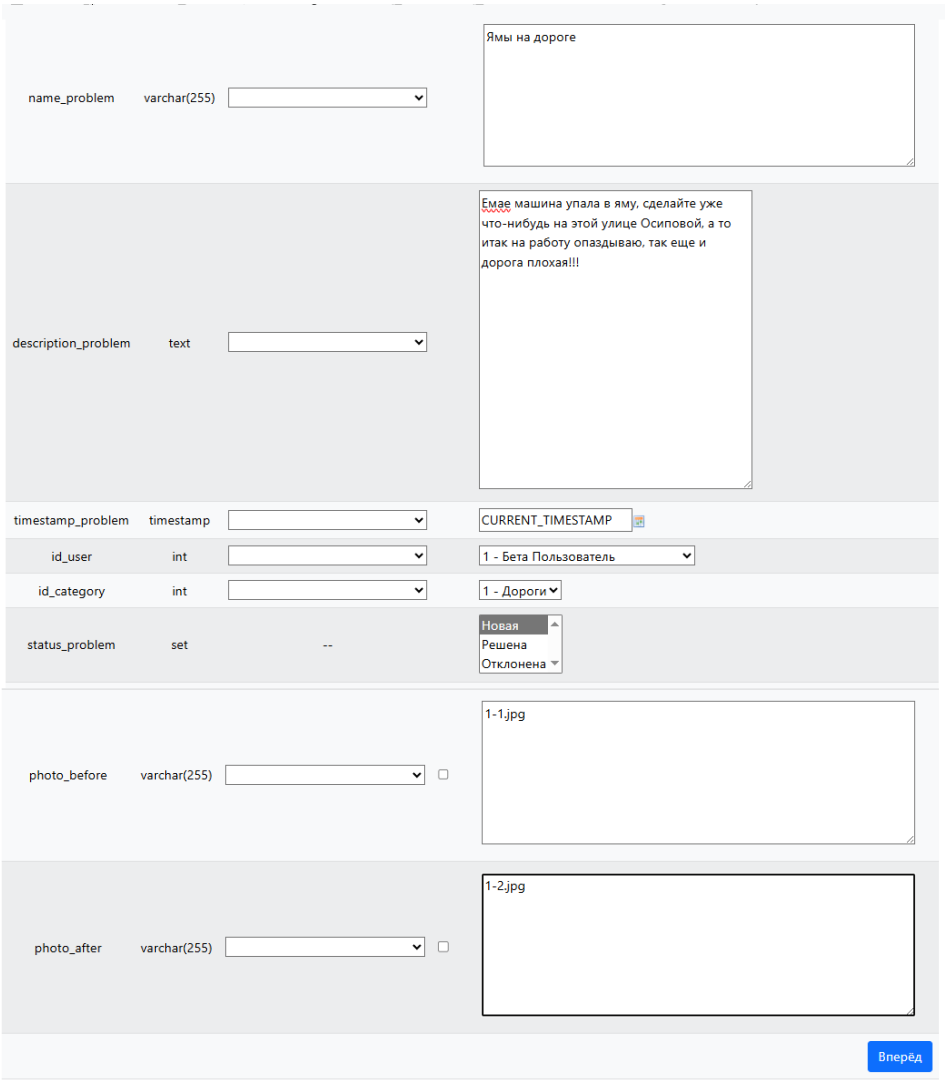


Создадим заявки вручную через базу данных. Откроем phpMyAdmin выбираем базу данных и таблицу problem. Операция вставки.



Заполняем поля таблицы. Допустим создадим проблему «Ямы на дороге», добавим какое-то описание, время по умолчанию, любой пользователь,

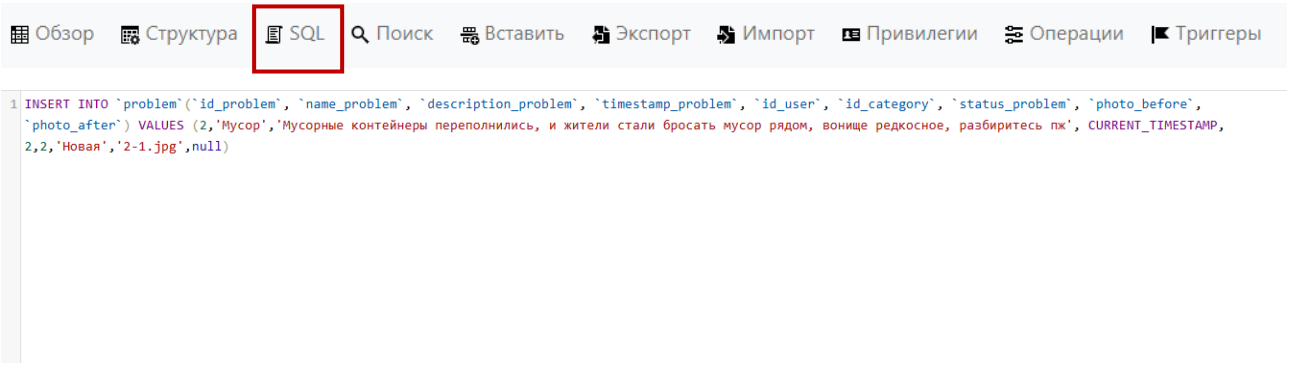
категория «Дороги», статус «Решена», фото до и фото после соответствуют вашим изображениям в подкаталоге uploads.



Нажимаем вперед. Запись автоматически добавилась в базу данных.

←T→	id_problem	name_problem	description_problem	timestamp_problem	id_user	id_category	status_problem	photo_before	photo_after
	1	Ямы на дороге	Емае, машина упала в яму!!!:(	2026-01-01 17:53:20	1	1	Новая	1-1.jpg	1-2.jpg

Вторую запись создадим с помощью SQL-запроса.



Нажимаем выполнить SQL-запрос. Вторая запись успешно добавилась в таблицу с заявками.

id_problem	name_problem	description_problem	timestamp_problem	id_user	id_category	status_problem	photo_before	photo_after
1	Ямы на дороге	Емае, машина упала в яму!!!:(	2026-01-01 17:53:20	1	1	Решена	1-1.jpg	1-2.jpg
2	Мусор	Мусорные контейнеры переполнились, и жители стали ...	2026-01-01 18:00:28	2	2	Новая	2-1.jpg	NULL

SiteController – контроллер, который принимает запрос пользователя и выступает посредником между двумя другими компонентами архитектуры: видом и моделью. Его задача — запросить данные из модели и передать их в вид, который должен быть вызван.

Откроем файл SiteController.php подкаталога controllers и добавим использование Yii.

```
use Yii;
use yii\filters\AccessControl;
use yii\web\Controller;
use yii\web\Response;
use yii\filters\VerbFilter;
use app\models\LoginForm;
use app\models\ContactForm;
use app\models\Problem;
```

Метод actionIndex генерирует страницу, соответствующую файлу представления index.php в соответствующем каталоге представлений вашего приложения Yii.

```
public function actionIndex()
{
    return $this->render('index');
}
```

Создадим переменную, которая будет хранить массив с элементами базы данных. Используетм статический метод find() модели Problem, который возвращает объект базы данных. Добавим условие фильтрации записей по полю status_problem, выбираются только записи с значением поля равным строке "Решена". Отсортируем записи по убыванию значения поля timestamp_problem. Ограничим количество возвращаемых записей до пяти.

```
$problems = Problem::find()->where(['status_problem' => 'Решена'])->orderBy(['timestamp_problem' => SORT_DESC])->limit(5)->all();
```

В метод render добавим второй аргумент, в который передается ассоциативный массив, где ключ массива соответствует имени переменной,

доступной в шаблоне представления, а значение — это фактическое значение переменной.

```
return $this->render('index', [
    'problems' => $problems,
]);
```

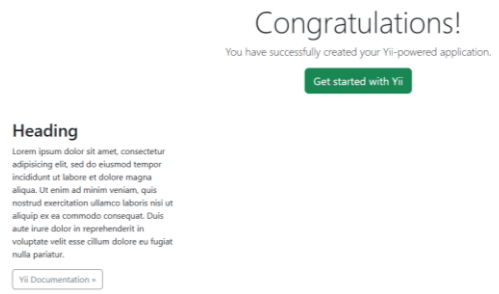
Отредактируем главную страницу проекта. Откроем index.php подкаталога site каталога views. Внутри тега div с классом row для каждой решенной заявки будем выводить образец кода.

```
<?php
    foreach($problems as $problem) {
        echo '
        <div class="col-lg-3 mb-3">
            <h2>Heading</h2>

            <p>Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do eiusmod tempor incididunt ut labore et
                dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip
                ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu
                fugiat nulla pariatur.</p>

            <p><a class="btn btn-outline-secondary" href="https://www.yiiframework.com/doc/">Yii Documentation &raquo;</a></p>
        </div>
        ';
    }
?>
```

Таким образом на главной странице должна отображаться одна запись, потому что столько записей имеет статус “решена”.



Для каждой решенной заявки выведем название и описание заявки, а также две фотографии.

```
foreach($problems as $problem) {
    echo '
    <div class="col-lg-3 mb-3 problem-info">
        <h2>'. $problem->name_problem .'</h2>

        <p>'. $problem->description_problem .'</p>

        <img src="uploads/'. $problem->photo_after .'" width="100%" alt="image" class="first-img">
        <img src="uploads/'. $problem->photo_before .'" width="100%" alt="image" class="second-img">
    </div>
    ';
}
```

Оформим минимальную стилизацию-анимацию для вывода фотографий до и после. Пропишем с помощью тега style.

```
<style>
    .problem-info {
        .second-img {
            display: none;
        }
    }

    .problem-info:hover {
        .first-img {
            display: none;
        }
        .second-img {
            display: block;
        }
    }
</style>
```

Скопируем модель Problem и переименуем ее в ProblemCreateForm.

ProblemCreateForm.php

```
class ProblemCreateForm extends Problem
```

Так как данный класс расширяет модель Problem, удалим все содержимое, оставив лишь правила валидации.

```
<?php

namespace app\models;

use Yii;

/**
 * This is the model class for table "problem".
 *
 * @property int $id_problem
 * @property string $name_problem
 * @property string $description_problem
 * @property string $timestamp_problem
 * @property int $id_user
 * @property int $id_category
 * @property string $status_problem
 * @property string|null $photo_before
 * @property string|null $photo_after
 *
 * @property Category $category
 * @property User $user
 */
class ProblemCreateForm extends Problem
{


    /**
     * {@inheritdoc}
     */
    public function rules()
    {
        return [
            [['photo_before', 'photo_after'], 'default', 'value' => null],
            [['status_problem'], 'default', 'value' => 'Новая'],
            [['name_problem', 'description_problem', 'id_user', 'id_category'], 'required'],
            [['description_problem', 'status_problem'], 'string'],
            [['timestamp_problem'], 'safe'],
            [['id_user', 'id_category'], 'integer'],
            [['name_problem', 'photo_before', 'photo_after'], 'string', 'max' => 255],
            [['id_category'], 'exist', 'skipOnError' => true, 'targetClass' => Category::class, 'targetAttribute' => ['id_category' => 'id_category']],
            [['id_user'], 'exist', 'skipOnError' => true, 'targetClass' => User::class, 'targetAttribute' => ['id_user' => 'id_user']],
        ];
    }



}
```

Сделаем обязательным полем загрузку файла с фотографией.

```
[['name_problem', 'description_problem', 'id_category' 'photo_before'], 'required'],
```

Поле с фотографией проверяет допустимые расширения и с максимальным размером 10 МБайт.

```
[['photo_before'], 'file', 'extensions' => 'png, jpg, jpeg, bmp', 'maxSize' => 10*1024*1024],
```

В файл create.php подкаталога lk скопируем содержимое формы из файла _form.php подкаталога problem.

```
<?php

use yii\helpers\Html;
use yii\widgets\ActiveForm;

/** @var yii\web\View $this */
/** @var app\models\Problem $model */

$this->title = 'Create Problem';
$this->params['breadcrumbs'][] = ['label' => 'Problems', 'url' => ['index']];
$this->params['breadcrumbs'][] = $this->title;
?>
<div class="problem-create">

    <h1><?= Html::encode($this->title) ?></h1>

    <div class="problem-form">

    <?php $form = ActiveForm::begin(); ?>

    <?= $form->field($model, 'name_problem')->textInput(['maxlength' => true]) ?>

    <?= $form->field($model, 'description_problem')->textarea(['rows' => 6]) ?>

    <?= $form->field($model, 'timestamp_problem')->textInput() ?>

    <?= $form->field($model, 'id_user')->textInput() ?>

    <?= $form->field($model, 'id_category')->textInput() ?>

    <?= $form->field($model, 'status_problem')->textInput(['maxlength' => true]) ?>

    <?= $form->field($model, 'photo_before')->textInput(['maxlength' => true]) ?>

    <?= $form->field($model, 'photo_after')->textInput(['maxlength' => true]) ?>

    <div class="form-group">
        <?= Html::submitButton('Save', ['class' => 'btn btn-success']) ?>
    </div>

    <?php ActiveForm::end(); ?>

</div>
```

Очистим форму удалив лишние поля, оставив только название проблемы с помощью метода textInput(), описание с помощью функции textarea(), выбор категории с помощью выпадающего списка dropDownList и массива с названием этих категорий, и поле для загрузки фотографии.

```
<?php $form = ActiveForm::begin(); ?>

<?= $form->field($model, 'name_problem')->textInput(['maxlength' => true]) ?>

<?= $form->field($model, 'description_problem')->textarea(['rows' => 6]) ?>

<?= $form->field($model, 'id_category')->dropDownList($categories) ?>

<?= $form->field($model, 'photo_before')->fileInput() ?>
```

Первая строка получает все записи из таблицы категорий (предположительно модели Category) и сохраняет их в переменную $categories. Метод find() создаёт объект-запрос, а метод all() возвращает массив всех записей.

Используя класс помощника массива ArrayHelper, преобразуя массив объектов (полученных ранее) в ассоциативный массив, где ключами будут значения столбца id_category, а значениями — значения столбца name_category.

```
$categories = Category::find()->all();
$categories = ArrayHelper::map($categories, 'id_category', 'name_category');
```

Добавим в отображение страницы массив с категориями.

```
return $this->render('create', [
    'model' => $model,
    'categories' => $categories,
]);
```

Поменяем модель на ProblemCreateForm().

```
$model = new ProblemCreateForm();
```

Импортируем необходимые библиотеки

```
use app\models\Category;
```

```
use app\models\ProblemCreateForm;
```

```
use yii\helpers\ArrayHelper;
```

Интерфейс формы для создании новой заявки реализован.

Create Problem

Название

Описание

Категория

Дороги

Photo Before Выбор файла Не выбран ни один файл

Save

В файле create.php подкаталога lk в модели ActiveForm пропишем специальный MIME-тип, обозначающий, что форма будет содержать бинарные данные, в данном случае загружать файл.

```
<?php $form = ActiveForm::begin(['options' => ['enctype' => 'multipart/form-data']]); ?>
```

В файле LkController в функции actionCreate() внутри вложенной условной конструкции оставим проверку загрузки.

Получаем загруженный файл с именем поля photo_before из модели. Класс UploadedFile помогает правильно обработать файл, поступивший через HTTP-запрос.

Сохраняем файл в папку /web/uploads/ приложения. Атрибут @app указывает на корневую директорию проекта.

```
if ($model->load($this->request->post())) {
    $model->photo_before = UploadedFile::getInstance($model, 'photo_before');
    $model->photo_before->saveAs('@app/web/uploads/' . $model->photo_before->baseName . '.' . $model->photo_before->extension);
    // return $this->redirect(['view', 'id_problem' => $model->id_problem]);
}
```

Обязательно прописываем импорт библиотеки.

```
use yii\web\UploadedFile;
```

Проверим сохранение фотографии в папку с медиафайлами.

Create Problem

Название

Яма

Описание

Яма

Категория

Дороги

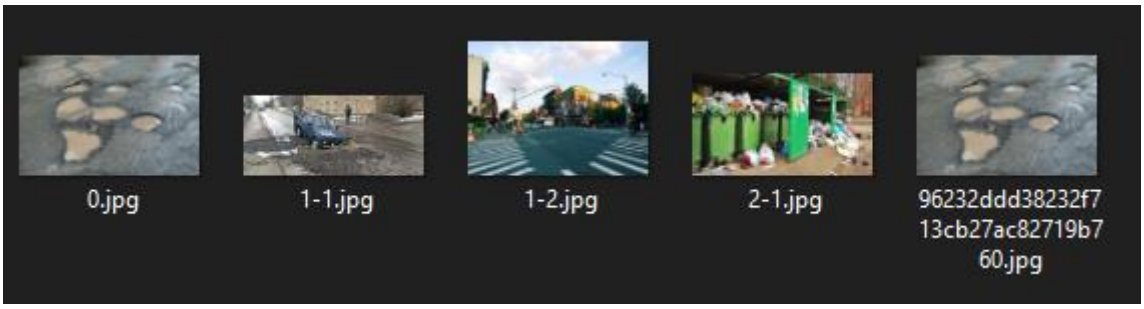
Photo Before Выбор файла 0.jpg

Фотография 0.jpg добавлена в подкаталог uploads каталога web.

Но у разных пользователей название фотографии может быть одинаковым, поэтому создадим новое имя файла путем хэширования базовой части имени файла и текущего времени с добавлением расширения файла.

```
$newFileName = md5($model->photo_before->baseName . '.' . $model->photo_before->extension . time());
$model->photo_before->saveAs('@app/web/uploads/' . $newFileName  . '.' . $model->photo_before->extension);
```

Проверим еще раз загрузив фотографию на сервер. В папке uploads добавится новая фотография.



Сохраняем данные в поля базы данных. Сохраняем модель и перенаправляем пользователя в личный кабинет.

```
$model->photo_before = $newFileName;
$model->id_user = Yii::$app->user->identity->id_user;
$model->save();
return $this->redirect(['/lk']);
```

Создадим новую запись для проверки.

Create Problem

Название

Яма

Описание

После дождя огромное озеро на дороге(

Категория

Дороги

Photo Before Выбор файла 0.jpg

Save

Запись с заявкой добавлена и отображается в личном кабинете.

2026-01-04 11:33:41	Яма	После дождя огромное озеро на дороге(	Дороги	Новая	